

< 텍스트 분석 >

** NLP이냐 텍스트 분석이냐?

- NLP : 인간의 언어를 이해하고 해석하는 데 중점
- 텍스트 분석 : 비정형 텍스트에서 의미 있는 정보를 추출하는 것에 중점
- 텍스트 분석의 기술
 - (1) 텍스트 분류 : 문서가 특정 분류 또는 카테고리에 속하는 것을 예측
 - (2) 감성 분석 : 주관적인 요소를 분석하는 기법
 - (3) 텍스트 요약 : 텍스트 내에서 중요한 주제나 중심 사상을 추출하는 기법
 - (4) 텍스트 군집화 : 비슷한 유형의 문서에 대해 군집화 수행

1. 텍스트 분석 이해

- 텍스트 분석 : 비정형 데이터인 텍스트를 분석하는 것
- 피처 벡터화/피처 추출 : 텍스트를 변환하는 것
- 텍스트 변환 방법 : BOW(Bag of Words) & Word2Vec

(1) 텍스트 분석 수행 프로세스

- 텍스트 사전 준비작업 : 클렌징 작업, 토큰화 작업, 제거 작업, 어근 추출 등의 텍스트 정규화 작업
- 피처 벡터화/추출 : 가공된 텍스트에서 피처를 추출 & 벡터 값 할당
- ML 모델 수립 및 학습/예측/평가 : ML모델 적용 & 학습/예측 및 평가

(2) 파이썬 기반의 NLP, 텍스트 분석 패키지

- NLTK : 파이썬의 가장 대표적인 NLP 패키지
- Gensim : 토픽 모델링 분야에서 가장 두각을 나타내는 패키지
- SpaCy : 뛰어난 수행 성능

2. 텍스트 사전 준비 작업(텍스트 전처리) – 텍스트 정규화

- 텍스트 정규화 : 텍스트를 데이터로 사용하기 위해 텍스트의 사전 작업을 수행
- 분류

(1) 클렌징

- 불필요한 문자, 기호 등을 사전에 제거하는 작업

(2) 텍스트 토큰화

** 토큰 : 의미있는 최소 단위

● 문장 토큰화

- 문장의 마지막을 뜻하는 기호(마침표, 개행문자)에 따라 분리하는 것이 일반적
- 각 문장이 가지는 시맨틱적인 의미가 중요한 요소로 사용될 때 사용
- `sent_tokenize()`

● 단어 토큰화

- 문장을 단어로 토큰화 하는 것
- 공백, 콤마(,), 마침표(.), 개행문자 등으로 단어 분리

- `word_tokenize()`

● n-gram

- 연속된 n개의 단어를 하나의 토큰화 단위로 분리
- 문장을 단어별로 토큰화 할 때, 문맥적의미가 무시되는 것을 해결하고자 도입

(3) 스톱 워드 제거

- 스톱 워드(Stop word) : 분석에 큰 의미가 없는 단어 (접속사, be동사 등)
- 의미 없는 단어들 제거

(4) Stemming과 Lemmatization

- 문법적 또는 의미적으로 변화하는 단어의 원형을 찾는 것
- Stemming
 - 원래 단어에서 일부 절차가 훼손된 어근 단어 추출
 - 일반적이거나 더 단순화된 방법 사용
 - NLTK 제공 Stemmer : Porter, Lancaster, Snowball, Stemmer
 - 변형된 단어의 정확한 원형을 찾지 못하고, 원형 단어에서 철자가 다른 어근으로 인식하는 경우 발생
- Lemmatization
 - 문법적인 요소와 더 의미적인 부분을 감안해 정확한 철자로 된 어근 단어 찾을
 - Stemming보다 정교
 - NLTK 제공 : WordNetLemmatizer
 - 정확한 원형 단어 추출을 위해 단어의 '품사'를 입력

3. Bag of Words – BOW

- 문서가 가지는 모든 단어(Words)를 문맥이나 순서를 무시
 - 일괄적으로 단어에 대해 빈도 값을 부여해 피쳐 값을 추출하는 모델
 - 장점 : 쉽고 빠른 구축
 - 단점 : 문맥 의미 반영 부족 (순서고려 X), 희소 행렬 문제
- ** 희소 행렬 : 행렬에서 대부분의 값이 0으로 채워지는 행렬

(1) BOW 피쳐 벡터화

- 피쳐 벡터화 : 텍스트가 특정 의미를 가지는 숫자형 값인 벡터 값으로 변환
- BOW 모델에서 피쳐 벡터화를 수행한다는 것
 - ➔ 모든 단어를 칼럼 형태로 나열
 - ➔ 해당 단어의 횟수나 정규화된 빈도를 값으로 부여하는 데이터로 변경
- BOW의 피쳐 벡터화 방법
 - 카운트 벡터화
- 각 문서에서 해당 단어가 나타내는 횟수(Count) 부여
 - TF-IDF
- 개별 문서에서 자주 나타나는 단어에 높은 가중치를 두되,
- 모든 문서에서 전반적으로 자주 나타나는 단어에 대해서 패널티 부여
 - ➔ 가중치의 균형을 맞춤
- 텍스트가 길고, 문서가 많은 경우 더 좋은 예측성능 보장 가능

(2) 사이킷런의 Count 및 TF-IDF 벡터화 구현 : CountVectorizer, TfidfVectorizer

- 사이킷런의 CountVectorizer 클래스
- 카운트 기반의 벡터화 구현
- 피쳐 벡터화만 수행하지 않고, 텍스트 전처리도 함께 수행
- 입력 파라미터를 설정해 동작

** 입력 파라미터

- max_df : 너무 높은 빈도수를 가지는 단어 피쳐 제외
- min_df : 너무 낮은 빈도수를 가지는 단어 피쳐 제외
- max_features : 추출하는 피쳐의 개수 제한, 정수로 값 지정
- stop_words : 추출에서 제외할 단어 지정
- n_gram_range : n_gram의 범위 설정, 튜플 형태

- analyzer : 피쳐 추출을 수행한 단위 지정
- token_pattern : 정규 표현식 패턴 지정
- tokenizer : 토큰화를 별도의 커스텀 함수로 이용시 적용

- 사용 방법



- 사이킷런에서 TF-IDF 벡터화

- TfidfVectorizer 클래스 이용
- 파라미터와 변환 방법은 CountVectorizer과 동일

(3) BOW 벡터화를 위한 희소 행렬

- 희소 행렬 : 대규모 행렬의 대부분의 값을 0이 차지하는 행렬
- 희소 행렬은 너무 많고 불필요한 0 값이 메모리 값에 할당되어 많은공간필요
→ 희소 행렬을 물리적으로 적은 메모리 공간을 차지할 수 있도록 변환
: COO 형식 & CSR 형식

(4) 희소 행렬 - COO 형식

- COO(Coordinate: 좌표) 형식
- 0이 아닌 데이터만 별도의 데이터 배열에 저장 & 그 데이터가 가리키는 행과 열의 위치를 별도의 배열로 저장하는 방식
- 희소 행렬 변환을 위해 주로 사이파이(Scipy) 이용

- 사이파이의 sparse 패키지는 희소 행렬 변환을 위한 다양한 모듈 제공
 - 사이파이의 sparse를 이용해 희소 행렬 변환을 COO로 수행
 - 1) 밀집 행렬을 넘파이의 ndarray 객체로 만들
 - 2) 위 밀집 행렬을 사이파이의 coo_matrix 클래스를 이용해 COO 형식의 희소 행렬로 변환

(5) 희소 행렬 – CSR 형식

- CSR (Compressed Sparse Row) 형식
- COO 형식이 행과 열의 위치를 나타내기 위해 반복적인 위치 데이터를 사용해야 하는 문제점을 해결
- 행 위치 배열 내에 있는 고유한 값의 시작 위치만 다시 별도의 위치 배열로 가지는 변환 방식
- COO 방식보다 메모리가 적게 들고 빠른 연산 가능
- 사이파이의 csr_matrix 클래스 이용
 - ➔ 0이 아닌 데이터 배열과 열 위치 배열, 행 위치 배열의 고유한 값의 시작 위치 배열을 생성 파라미터로 입력

4. 감성 분석

(1) 감성 분석 소개

- 감성 분석 (Sentiment Analysis)
- 문서의 주관적인 감성/의견/감정/기분 등을 파악하기 위한 방법
- 문서 내 텍스트가 나타내는 여러 가지 주관적인 단어와 문맥 기반, 감성수치계산
- 지도 학습 : 학습 데이터와 타깃 레이블 값을 기반으로 감성 분석 학습 수행 후 다른 데이터의 감성 분석 예측
- 비지도 학습 : 'Lexicon'이라는 일종의 감성 어휘 사전 이용

(2) 지도학습 기반 감성 분석 실습

- 데이터 클렌징
- 감상평(Review) 텍스트를 피쳐 벡터화
- ML 분류 알고리즘을 적용해 예측 성능 측정
- 예측 성능 평가

(3) 비지도학습 기반 감성 분석 소개

- 비지도 감성 분석은 Lexicon 기반으로 수행
- 감성 사전을 구현한 대표격은 NLTK 패키지
- NLTK에서 제공하는 WordNet 모듈 : 방대한 영어어휘사전, 시맨틱 분석 제공
 - 시맨틱 : 문맥상 의미
- NLP 패키지는 시맨틱을 프로그램적으로 인터페이스 할 수 있는 방법 제공
- WordNet : 각각의 품사로 구성된 개별 단어를 Synset이라는 개념으로 표현
 - Synset : 단어가 가지는 문맥, 시맨틱 정보를 제공하는 WordNet의 핵심개념
- NLTK의 감성 사전은 예측 성능이 좋지 못하다는 단점을 가짐

** 대표적인 감성 사전

- SentiWordNet : 감성 단어 전용의 WordNet 구현
- VADER : 소셜 미디어의 텍스트에 대한 감성 분석을 제공하기 위한 패키지
- Pattern : 예측 성능 측면에서 가장 주목받는 패키지

(4) SentiWordNet을 이용한 감성 분석

- 문서(Document)를 문장(Sentence) 단위로 분해
- 다시 문장을 단어(Word) 단위로 토큰화하고 품사 태깅
- 품사 태깅된 단어 기반으로 synset 객체와 senti_synset 객체를 생성
- senti_synset 객체에서 긍정 감성/부정 감성 지수를 구하고 이를 모두 합산해 특정 임계치 값 이상일 때 긍정 감성 으로, 그렇지 않을 때는 부정 감성으로 결정

(5) VADER를 이용한 감성 분석

- VADER : 소셜 미디어의 감성 분석 용도로 만들어진 룰 기반의 Lexicon
- SentimentIntensityAnalyzer 클래스 이용
- 수행 순서
 - 1) SentimentIntensityAnalyzer 객체 생성
 - 2) 문서별로 polarity_scores() 메서드 호출해 감성 점수 구함
 - 3) 감성 점수가 특정 임계값 이상이면 긍정, 그렇지 않으면 부정으로 판단